Contents

1 General **1**
- 1.1 Compilation . . . 1
- 1.2 Template . . . 1

2 Miscellaneous **1**
- 2.1 Bounds . . . 1

3 Number Theory **2**
- 3.1 Binary Exponentiation . . . 2

4 Bits **2**
- 4.1 Gray Code . . . 2

5 Graphs **2**
- 5.1 Dijkstra . . . 2

6 Data Structures **2**
- 6.1 Segment Tree . . . 2

7 Math **3**
- 7.1 Linear Sieve . . . 3
- 7.2 FFT . . . 3
- 7.3 NTT . . . 3
- 7.4 Multiply . . . 4

8 Strings **4**
- 8.1 KMP . . . 4
- 8.2 Manacher . . . 5

1 General

1.1 Compilation

```
g++ problem.cpp -std=c++23 -O2 -Wall -Wextra -Wshadow -Wconversion -Wfloat-equal -Wduplicated-cond -Wlogical-op -o
    r && ./r < in.txt > out.txt
```

1.2 Template

```
#include<bits/stdc++.h>
#define debug(x) cerr<<#x<<":"<<x<<endl;
typedef long long ll;
using namespace std;
int main(){
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    return 0;
}
```

2 Miscellaneous

2.1 Bounds

```
int lowerBound(vector<int> &v,int target){
  int l=0;
  int r=v.size();
  while(l<r){
    int mid=l+(r-l)/2;
    /*To change to upper use <= instead of <*/
    if(v[mid]<target)
        l=mid+1;
    else
        r=mid;
  }
  return l;
}
```

3 Number Theory

3.1 Binary Exponentiation

```
ll binpow(ll a, ll b){
    ll res=1;
    while(b){
        if(b&1) res*=a;
        a*=a;
        b>>=1;
    }
    return res;
}
```

4 Bits

4.1 Gray Code

```
int g (int n) {
    return n ^ (n >> 1);
}
int rev_g (int g) {
  int n = 0;
  for (; g; g >>= 1)
    n ^= g;
  return n;
}
```

5 Graphs

5.1 Dijkstra

```
typedef pair<ll,ll> Esp;
void dijkstra(){
    vector<vector<Esp>> graph(n+1);
    vector<ll> dist(n+1,INF);
    priority_queue<Esp,vector<Esp>,greater<Esp>> pq;
    pq.push({0,1});
    dist[1]=0;
    while(!pq.empty()){
        auto [n_dist,node]=pq.top(); pq.pop();
        if(n_dist>dist[node]) continue;
        for(auto [v2,cost]:graph[node]){
            if(dist[v2]>n_dist+cost){
                dist[v2]=dist[node]+cost;
                pq.push({dist[v2],v2});
            }
        }
    }
}
```

6 Data Structures

6.1 Segment Tree

```
struct Node{
    Node operator+(const Node &other) const{
        //merge
    }
};
struct SegmentTree {
    int n;
    vector<Node> st;
    SegmentTree(int n, vector<ll>& v) {
        this->n = n;
        st.assign(2 * n, Node());
        for (int i = 0; i < n; i++) st[n + i] = Node(v[i]);
        for (int i = n - 1; i > 0; i--)
            st[i] = st[i << 1]+st[i << 1 | 1];
    }
    void update(int pos, ll value) {
        pos += n;
        st[pos] = value;
        for (int i = pos >> 1; i > 0; i >>= 1) {
            st[i] = st[i << 1]+st[i << 1 | 1];
        }
    }
    Node query(int l, int r) {
```

```
        Node resL = Node(), resR = Node();
        for (l += n, r += n; l < r; l >>= 1, r >>= 1) {
            if (l & 1){
                resL = resL+ st[l]; l++;
            }
            if (r & 1){
                r--; resR =st[r]+ resR;
            }
        }
        //[l,r)
        return resL+ resR;
    }
};
```

7 Math

7.1 Linear Sieve

```
vector<int> mindiv;
vector<int> primes;
void sieve(int N) {
    mindiv = vector<int>(N+1, 0);
    primes = vector<int>();
    for(int i=2; i <= N; ++i) {
        if(mindiv[i] == 0) {
            mindiv[i] = i;
            primes.push_back(i);
        }
        for(int j=0; j < primes.size() and primes[j] <= mindiv[i] and primes[j]*i <= N; ++j) {
            mindiv[i*primes[j]] = primes[j];
        }
    }
}
```

7.2 FFT

```
using cd = complex<double>;
const double PI = acos(-1);

int reverse(int num, int lg_n){
    int res = 0;
    for (int i = 0; i < lg_n; i++){
        if (num & (1 << i))
            res |= 1<<(lg_n-1-i);
    }
    return res;
}

void fft(vector<cd> & a, bool invert) {
    int n = a.size();
    int lg_n = 0;
    while ((1 << lg_n) < n) lg_n++;

    for (int i = 0; i < n; i++) {
        if (i < reverse(i, lg_n))
            swap(a[i], a[reverse(i, lg_n)]);
    }
    for (int len = 2; len <= n; len <<= 1) {
        double ang = 2 * PI / len * (invert ? -1 : 1);
        cd wlen(cos(ang), sin(ang));
        for (int i = 0; i < n; i += len) {
            cd w(1);
            for (int j = 0; j < len / 2; j++) {
                cd u = a[i+j], v = a[i+j+len/2] * w;
                a[i+j] = u + v;
                a[i+j+len/2] = u - v;
                w *= wlen;
            }
        }
    }
    if (invert) {
        for (cd & x : a) x/=n;
    }
}
```

7.3 NTT

```
const int mod = 7340033;
const int root = 5;
const int root_1 = 4404020;
const int root_pw = 1 << 20;
/*
    const int mod = 998244353;
```

```
    const int root = 31;
    const int root_1 = 128805723;
    const int root_pw = 1 << 23;

    const int mod = 1004535809;
    const int root = 12289;
    const int root_1 = 3133564925;
    const int root_pw = 1 << 21;

    const int mod = 469762049;
    const int root = 2187;
    const int root_1 = 410692747;
    const int root_pw = 1 << 26;
*/
void fft(vector<int> & a, bool invert) {
    int n = a.size();
    for (int i = 1, j = 0; i < n; i++) {
        int bit = n >> 1;
        for (; j & bit; bit>>=1) j ^= bit;
        j ^= bit;

        if(i<j) swap(a[i], a[j]);
    }

    for (int len = 2; len <= n; len <<= 1) {
        int wlen = invert ? root_1 : root;
        for (int i = len; i < root_pw; i <<= 1)
            wlen = (int)(1LL * wlen * wlen % mod);

        for (int i = 0; i < n; i += len) {
            int w = 1;
            for (int j = 0; j < len / 2; j++) {
                int u = a[i+j], v = (int)(1LL * a[i+j+len/2] * w % mod);
                a[i+j] = u + v < mod ? u + v : u + v - mod;
                a[i+j+len/2] = u - v >= 0 ? u - v : u - v + mod;
                w = (int)(1LL * w * wlen % mod);
            }
        }
    }
    if (invert) {
        int n_1 = inverse(n, mod);
        for (int & x : a)
            x = (int)(1LL * x * n_1 % mod);
    }
}
```

7.4 Multiply

```
vector<int> multiply(vector<int> const& a, vector<int> const& b) {
    vector<cd> fa(a.begin(), a.end()), fb(b.begin(), b.end());
    int n = 1;
    while (n < a.size() + b.size())
        n <<= 1;
    fa.resize(n);
    fb.resize(n);

    fft(fa, false);
    fft(fb, false);
    for (int i = 0; i < n; i++)
        fa[i] *= fb[i];
    fft(fa, true);
    // For nnt just return fa
    // and apply MOD to fa[i]
    // If you want return result
    // Clean the right zeros
    vector<int> result(n);
    for (int i = 0; i < n; i++)
        result[i] = round(fa[i].real());

    return result;
}
```

8 Strings

8.1 KMP

```
vector<int> genLps(string &cad){
    int len=0;
    int j=1;
    int n=cad.size();
    vector<int> lps(n);
    lps[0]=0;
    while(j<n){
        if(cad[len]==cad[j]){
            len++;
            lps[j]=len;
```

```
            j++;
        }
        else{
            if(len!=0) len=lps[len-1];
            else{
                lps[j]=0;
                j++;
            }
        }
    }
    return lps;
}
vector<int> kmp(string &text, string &pat){
    vector<int> lps=genLps(pat);
    vector<int> ans;
    int i=0;
    int j=0;
    int n=text.size();
    while(i<n){
        if(pat[j]==text[i]){
            i++; j++;
            if(j==pat.size()){
                ans.push_back(i-j);
                j=lps[j-1];
            }
        }
        else{
            if(j!=0) j=lps[j-1];
            else i++;
        }
    }
    return ans;
}
```

8.2 Manacher

```
vector<int> manacher_odd(string s) {
    int n = s.size();
    s = "$" + s + "^";
    vector<int> p(n + 2);
    int l = 0, r = 1;
    for(int i = 1; i <= n; i++) {
        if(i <= r) {
            p[i] = min(r-i, p[l + (r - i)]);
        }
        while(s[i - p[i]] == s[i + p[i]]) {
            p[i]++;
        }
        if(i + p[i] > r) {
            l=i-p[i], r=i+p[i];
        }
    }
    return vector<int>(begin(p) + 1, end(p) - 1);
}
```